Clautolisp Debugger — Remote CAD Backend Addendum

Debugging AutoLISP executed in a remote BricsCAD/AutoCAD engine, driven from alfe

Addendum to clautolisp-debugger-spec v0.1

June 1, 2026

Contents

1 0. Scope, status, and the one tenet that changes

This is a normative addendum to `clautolisp-debugger-spec.org` ("the parent spec"; section references like §9.4 are to it). It specifies how to run the parent debugger when the AutoLISP program under test is **not** evaluated by clautolisp in the debugger's own image, but is shipped over a protocol to a **remote native AutoLISP engine** inside BricsCAD or AutoCAD and evaluated there — the alfe deployment.

This is exactly the case the parent spec defers in **Part VIII**:

> "*Remote debugging.* Same-image debugger and application thread is the design [Strandh2020 §4]. The Emacs UI's RPC is the only network-capable surface. Cross-machine debugger application is explicitly not pursued."

Adopting the alfe deployment therefore **overturns one foundational tenet** of the parent spec, and only one:

> **Parent §1.1:** "The debugger and the AutoLISP program run in the same Common Lisp image, in different threads … There is no wire protocol between debugger and application; communication is via shared Lisp data plus a bordeaux-threads queue and semaphore."

In the alfe deployment the application runs in a **different process on a possibly different machine**, in the CAD engine's **native AutoLISP** (not clautolisp). The debugger application channel **must** become a wire protocol. Everything else in the parent spec is preserved by construction (see the thesis in 2.3).

1.1 0.1 The single most important consequence

The parent's claim that the debugger inspects a snapshot "*synchronously, since it has direct CL access — no message needed*" (§8.3) **does not hold remotely**. Every snapshot field beyond what is eagerly serialized, every `cmd-eval`, `cmd-set-variable`, and every inspector descent is an **RPC round-trip into the paused backend**. The parent's deferred "async eval-in-frame" (§8.3, `:eval-result`) is therefore **mandatory** here, not optional. Design accordingly: make poll-point traversal and stepping incur **no** round-trips (they stay on the backend), and pay round-trips only on a hit and during interactive inspection.

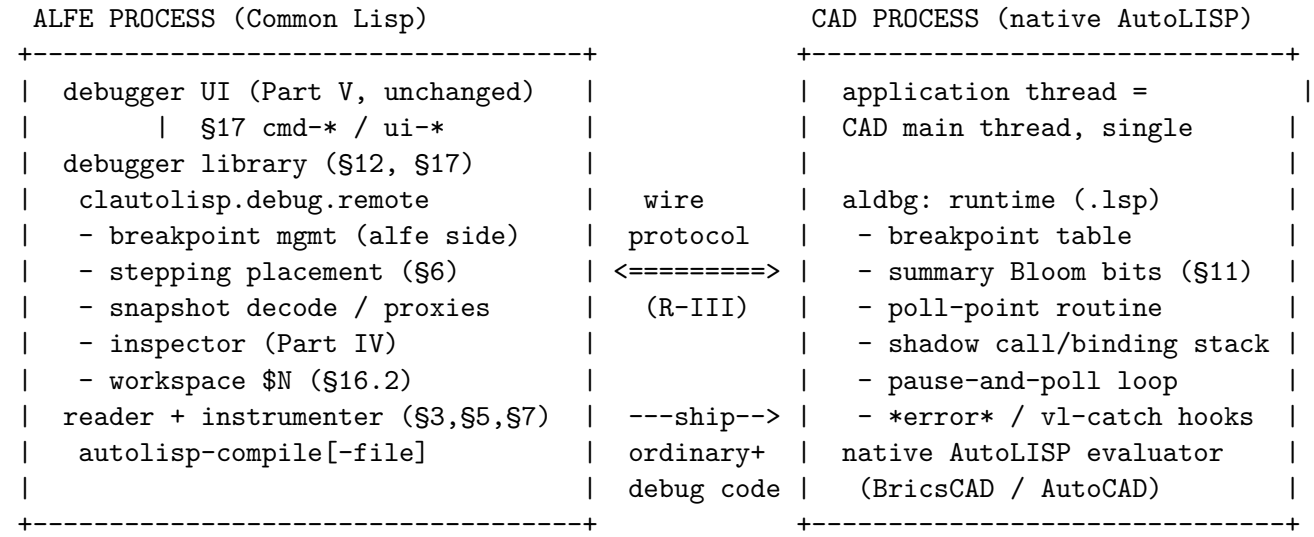
1.2 0.2 Naming

- `clautolisp.debug` — the parent's in-image application debugger contract package (§8). Unchanged as an *interface*; re-implemented over the wire here.
- `clautolisp.debug.remote` — new CL package (in alfe): an implementation of the §12/§17 debugger-library API whose Part-III channel is wire-backed.

- `aldbg:` — new **AutoLISP** runtime package on the backend (a `.lsp` program). It is the native-AutoLISP re-implementation of the *application side* of the Part-III contract. It is plain AutoLISP, not CL.

2 1. Architecture mapping

2.1 1.1 Topology



2.2 1.2 Where each parent component lives

Parent component	Alfe deployment location
Reader with source tracking (§3)	alfe (CL). Unchanged.
Compiler / instrumenter (§5, §7)	alfe (CL); **retargeted to emit AutoLISP** (R-II).
Debug metadata: form-id pos, kinds, parent -map, bound-names (§7)	alfe (CL). Stays in alfe; backend gets numbers.
Debugger library: breakpoints, stepping, snapshot logic, §12/§17 API	alfe (CL), `clautolisp.debug.remote`.
Inspector (Part IV)	alfe (CL); accessors evaluated on backend (R-IV).
Workspace $N (§16.2)	alfe (CL). Unchanged (debugger-private).
UIs (Part V)	alfe / Emacs. Unchanged.
Application side of Part III (§4, §8 app half, poll-point routine §11, snapshot producer §9, **error** path §10)	**backend, `aldbg:`, native AutoLISP** (R-III).
The actual program + CAD side effects	backend native AutoLISP engine.

2.3 1.3 Thesis: re-implement Part III, amend the rest locally

1. **Part I (Foundations).** Amended: §1.1's same-image/no-wire claim is replaced by 2. §1.2 threads: the backend is **single-threaded** (the CAD main thread); "application thread" = "the backend session." The "debugger thread" is in alfe. §1.3's three reasons to not reuse the host debugger hold a fortiori (the host here is a CAD engine with only the in-process IDE debuggers, which a remote client cannot reach).

2. **Part II (Compilation/instrumentation).** Retargeted: "two CL function bodies selected by entry point" (§5) becomes "two **AutoLISP** function bodies" (3.2). Poll-point calls (§5.3) are AutoLISP calls into `aldbg:`.

3. **Part III (the debug protocol). Replaced** by a wire-backed implementation (4). The *message kinds* of §8.2/§8.3 are preserved and extended; the *transport* changes from a `bordeaux-threads` queue to a framed wire channel; the *snapshot* changes from live CL objects to a serialized form with proxies (4.4).

4. **Part IV (Inspector).** Preserved; descents that need live CAD handles become backend evals of the accessor S-expressions the inspector already produces (5).

5. **Part V (UIs).** Preserved unchanged at the API level; only latency/async notes apply (7).

6. **Part VI (Sessions).** Amended for one backend thread and per-document sessions (8).

The §12 and §17 signatures **do not change**. A UI written against the parent spec runs unmodified against `clautolisp.debug.remote`.

3 R-II. Compilation and instrumentation, retargeted to AutoLISP

3.1 R-II.1 Why the instrumenter stays in alfe

Two reasons, the second decisive:

1. alfe already owns the source-tracking reader (§3) and the instrumentation logic (§5, §7). Reusing them is free.

2. The native AutoLISP `read` primitive returns only the **first** expression of a string and exposes no continuation cursor, so walking a multi-form source **on the backend** is impractical. alfe holds the forms it is about to send and has a full reader. **All** source-to-instrumented transformation therefore happens in alfe; the backend never parses user source. It receives the `aldbg:` runtime once, then pairs of already-emitted bodies plus numeric tables.

3.2 R-II.2 Two AutoLISP bodies; Fjeld propagation in source

The parent emits `foo/ordinary` and `foo/debug` as CL functions behind an entries vector (§5.1). Native AutoLISP has no call-site entry-point dispatch: `(foo …)` resolves `foo` by symbol. We reproduce the parent's two mechanisms — two bodies (Raskin) and debugging-ness propagating down the call chain (Fjeld) — **at the AutoLISP source level**:

For `(defun FOO (a b / z) BODY)` the alfe instrumenter emits two AutoLISP defuns:

```
;; ordinary body: optimised, no poll points, calls ordinary callees
(defun FOO~ord (a b / z) <BODY, calls BAR~ord, BAZ~ord, …>)

;; debugging body: poll points; calls the *debug* bodies of callees
(defun FOO~dbg (a b / z)
  (aldbg:enter '<fid:FOO> (list (cons 'A a) (cons 'B b) (cons 'Z nil)))
  (aldbg:pp '<fid:FOO> 0 :function-entry)
```

```
(aldbg:exit '<fid:FOO>
  (progn
    (aldbg:pp '<fid:FOO> 1 :before) <instrumented BODY, calls BAR~dbg …>
    (aldbg:pp '<fid:FOO> 1 :after))))
```

Propagation: in `FOO~dbg`, every call to an *instrumentable* callee `BAR` is rewritten to `BAR~dbg`; in `FOO~ord` it is `BAR~ord`. This is the source-level analog of Fjeld's entry points and yields the parent's **omnipresence**: entering through a debug body keeps you in debug bodies all the way down, with no per-function "compile for debug?" decision.

Activation. While a debug session is active, the public symbol `FOO` is bound to `FOO~dbg` (so top-level invocation from alfe and any un-rewritten reference enters instrumented code); `FOO~ord` is retained for the two-versions toggle and for code explicitly excluded from debugging. `aldbg:use-ordinary` / `aldbg:use-debug` swap the public binding. Because the backend has a **single** application thread (no concurrent production run to keep at full speed), the per-call-site dispatch of §5.1 is unnecessary; "which body is installed under the public name" suffices. (The parent needed per-thread/per-site dispatch only to keep *other* threads fast, §5/§23 — a concern that vanishes with one backend thread; see 8.)

Capturing an already-loaded native body. When source is unavailable but a function is already defined natively, its body can be recovered as a list only if it was defined with `defun-q` (Visual LISP's `defun-q=/=defun-q-list-ref=/=defun-q-list-set` preserve the internal list representation; plain compiled `defun` does not). Since alfe normally has the source, this is a fallback path only. Verify the `defun-q-list-*` accessor names per product/version.

3.3 R-II.2b The instrumentation boundary: only alfe-loaded code is debuggable

A foundational consequence of 3 (all source-to-instrumented transformation happens in alfe; the backend never parses user source): **a function is debuggable only if alfe produced its `~dbg` body.** In a real CAD deployment, **most** code reaching the engine was loaded by the CAD itself — `ACADDOC.lsp` / `on_doc_load`, `APPLOAD` and the Startup Suite, `(autoload …)` and `(autoarxload …)`, `VLX/FAS` bundles, partial CUIx LISP, `S::STARTUP`, `(load …)` calls inside already-running native code. None of it passed through alfe's reader/instrumenter, so none of it has `~ord=/=~dbg` bodies, no function-id, no form-id table, and no `aldbg:pp` poll points. The debugger **cannot** set source breakpoints, step, or build a shadow frame inside such a function;

it can only step *over* it (it is on the same footing as a built-in `SUBR`, R-X.1), and observe it post-mortem if it errors under an instrumented caller.

Therefore, **normatively**:

- `REQ-LOAD1:` To debug a function `FOO`, the programmer **MUST** (re)load its defining source **through alfe/clautolisp** (`autolisp-compile=/=autolisp-compile-file`, R-II.5) during — or before — the debug session. alfe then instruments it and installs `FOO~dbg` under the public name `FOO` (R-II.2 *Activation*), **shadowing** any natively-loaded definition for the duration of the session.

- `REQ-LOAD2:` Because instrumented `FOO~dbg` calls are rewritten to call the `~dbg` bodies of its callees (Fjeld propagation, R-II.2), a callee `BAR` that was **not** loaded through alfe has no `BAR~dbg`; the rewrite **MUST** fall back to the public `BAR` (the native body) and the debugger steps *over* it. Debugging-ness therefore propagates only across the subgraph of functions alfe has instrumented; it stops at the boundary of natively-loaded code.

- The UI **SHOULD** make this boundary visible: mark frames/functions with no alfe-side debug metadata as *uninstrumented* (no breakpoints settable, step-in degrades to step-over), so the programmer knows to re-load that source through alfe if they want to step into it.

- The `defun-q` capture path (R-II.2 above) is the **only** exception, and a best-effort fallback: a natively-loaded `defun-q` function's list body can be recovered and re-instrumented without its original source file. Plain `defun=/compiled/=VLX` bodies cannot, and stay uninstrumented.

This is the remote-deployment analog of the parent's omnipresence caveat: in the parent spec *everything* clautolisp evaluates is instrumentable because clautolisp is the evaluator; here the evaluator is the foreign CAD engine, so "omnipresence" holds only over the code alfe shipped. The teardown step (R-VI §24) restores the native definitions by swapping the public names back.

3.4 R-II.3 Optimisations disabled in the debug body (§5.2), AutoLISP form

§5.2's list maps cleanly: keep each instrumented sub-form's result live until its `:after` poll point; no motion across poll points; **no tail-call elimina-**

tion (AutoLISP does not guarantee TCO anyway, so the debug body simply must not be hand-rewritten to reuse frames). §5.2's "no type-inference across poll points" is moot — native AutoLISP holds values as tagged data and variable access is symbol lookup, so write-safety (§9.5) holds unconditionally.

3.5 R-II.4 Poll-point calls are AutoLISP (§5.3, §11.3)

Each poll point is a call `(aldbg:pp fid k when)`. `fid` is a **stable integer function-id assigned by alfe** (native AutoLISP has no `sxhash`; alfe-assigned ids replace it, which also makes the Bloom hash deterministic). `k` is the dense form-id; `when` ∈ `{:before :after :function-entry :function-exit}`. The parent's interpreter variant (§11.3) is subsumed: the backend's native engine plays the role of "the compiler", and alfe's instrumenter is what emits the two shapes — there is no separate backend `*debug-mode*` interpreter to maintain.

3.6 R-II.5 `autolisp-compile` and `autolisp-compile-file`

These are the alfe-side entry points the question asks for. They reuse §3/§5/§7 but target AutoLISP and install on the backend.

```
(clautolisp.debug.remote:autolisp-compile FORM &key (debug t)) → fn-handle
(clautolisp.debug.remote:autolisp-compile-file PATH &key (debug t)) → file-handle
```

`autolisp-compile-file`:

1. Read all top-level forms with the §3 source-tracking reader.
2. Assign function-ids and dense form-ids; build the §7 `function-debug-metadata` (`form-id→position`, `form-id→kind`, `parent-form-map`, `bound-names`, `source-text`) **in alfe**.
3. Emit, per function, the `~ord` and `~dbg` AutoLISP bodies (3.2).
4. Ensure the `aldbg:` runtime is loaded on the backend (once per session).
5. Transmit both bodies and register the function-id with the backend.
6. Return a handle alfe uses to toggle versions, place breakpoints (§12), and resolve `:hit` events to source.

The backend stores only: the two definitions, the function-id, and (per breakpoint) numeric table entries. Source positions, kinds, and the parent-form-map needed to **place** step breakpoints live in alfe.

4 R-III. The wire-backed debug protocol

This replaces Part III's transport while preserving its message vocabulary.

4.1 R-III.1 Backend per-session state (`aldbg:`), the §4/§8.1 analog

The parent's `thread-debug-info` (§8.1) is re-created in AutoLISP globals, document-namespaced (9):

- `aldbg:*debug-flag*` — master on/off; clear `aldbg:pp` does one global read and one branch, the native analog of §11.1's fast path.
- `aldbg:*breakpoints*` — table keyed by `(fid . form-id)`, value `(when steady? cond-fid action-fid hits)`. Two-level (assoc by `fid`, vector by `form-id`) to keep the hot path allocation-light.
- `aldbg:*summary*` — the §11.2 **Bloom filter**, 1024 bits over a vector of fixnums manipulated with `logand=/=logior=/=lsh` (AutoLISP has these). Index = `(rem (+ (* fid 1009) form-id) 1024)`. Set on add; rebuilt lazily on shrink (false positives tolerated, false negatives not — §11.2 unchanged).
- `aldbg:*current-pp*` — current `(fid . form-id)`, maintained by `pp`.
- `aldbg:*frames*` — the shadow call/binding stack (4.4).
- `aldbg:*catch-stack*` — active `vl-catch-all-apply` frames (§10.2).
- `aldbg:*volatile*` — volatile breakpoints for stepping (§6).

4.2 R-III.2 The poll-point routine on the backend (§11.1) — no round-trip unless it fires

```
(defun aldbg:pp (fid k when / key)
  (if aldbg:*debug-flag*                                ; (a) fast path
    (if (aldbg:bloom-test fid k)                        ; (b) Bloom
```

```
(progn
  (setq key (aldbg:find-bp fid k when))            ; (c) exact
  (if key (aldbg:handle-hit key when))))) ; only here do we touch the wire
```

Crucially, `aldbg:pp` consults **local** backend tables. Traversing thousands of poll points, and "continue/run to next volatile breakpoint", cost **zero** wire traffic. The wire is used only when a breakpoint actually fires (`aldbg:handle-hit`). This is what carries the parent's §11 performance thesis across the network — the Bloom filter matters **more** here, because the alternative (asking alfe at every poll point) would be fatal.

4.3 R-III.3 The pause loop (the §8.3 "blocked on inbound queue", now over the wire)

`aldbg:handle-hit` and the `*error*` path call `aldbg:pause`:

1. Build a snapshot (4.4) and send a `:hit` (or `:unhandled-error` / `:caught-error`) message over the wire.
2. **Block reading the wire**, dispatching debug commands one at a time: inspection/eval/breakpoint-table commands are answered inline (each is the round-trip that was "free synchronous CL access" in §8.3); a resume command (`:continue`, or "set these volatile breakpoints and continue" for stepping) **returns** from `aldbg:pause`, letting the suspended native evaluation proceed.

This blocks the CAD main thread exactly as the in-process IDE debuggers freeze the editor at a breakpoint — expected behaviour, not a defect. Requirements:

- `REQ-T1:` The transport must be **synchronously readable from AutoLISP while a top-level evaluation is suspended** (a socket/pipe/file the backend can read mid-evaluation). A transport that can deliver the next request only after the previous response is emitted cannot support mid-evaluation pause; with such a transport, only post-mortem debugging (break-on-error with a reconstructed backtrace) is possible, not live breakpoints/stepping.
- `REQ-T2:` Framing must distinguish a **top-level result** from **debug events** and **command replies**, because debug traffic is nested inside a not-yet- finished top-level `:eval` request (4.5).

- `REQ-T3:` Reentrancy guard: CAD reactors (`vlr-*`) may fire during a pause; `aldbg:pause` sets a guard so reactor callbacks do not re-enter the debugger.

4.4 R-III.4 The snapshot, serialized, with proxies for opaque values

The parent snapshot (§9.2) is a bundle of live CL objects. Remotely it must be **serialized**. The `defstruct snapshot` fields map as follows.

4.4.1 R-III.4.1 The shadow binding/call stack (§9.1, §9.3, §9.4)

Native AutoLISP does not expose its binding stack to a remote client, so `aldbg:` maintains its own, populated by instrumentation:

- `aldbg:enter` (emitted at each debug-body entry, 3.2) pushes a frame recording `bindings-introduced` as `(symbol . old-value-or-:unbound)` for each formal and /-local, in binding order (§7 `bound-names`, §9.3). The "old value" is read **before** the native binding shadows it, so the chain of shadowed bindings (§9.4) is reconstructable.
- Instrumented `setq` updates the relevant frame's recorded value.
- `aldbg:exit` pops the frame.

From this registry `aldbg:` can produce `call-stack`, `binding-stack`, and `visible-names` (§9.2) by the innermost-first walk §9.2 specifies. This delivers §9.4's signature feature — showing *all* bindings of a name including currently shadowed ones, and **writing a shadowed entry** — because the shadowed values are held in our registry and are restored on frame exit; mutating the registry entry changes what gets restored. §9.3's `with-frame-bindings` (eval in an outer frame) is implemented on the backend by temporarily restoring that frame's binding view from the registry, evaluating under `vl-catch-all-apply`, then re-establishing the inner bindings (always restore, even on error).

4.4.2 R-III.4.2 Value serialization and proxies

- **Serializable values** (integer, real, string, symbol, `nil`, and lists thereof) are sent inline in the snapshot via the protocol's printed-value mechanism.

- **Opaque CAD handles** (ename, selection set, file descriptor, VLA object) cannot be reconstructed as live objects in alfe. The snapshot carries a **proxy**: `(:proxy backend-handle-id type-tag printed-preview)`. The backend holds `backend-handle-id → live handle` in a per-pause table.
- This dovetails with the inspector (§15.1, 5): the inspector's accessors (`(entget _)`, `(ssname _ n)`, `(vla-get-prop _)`) are evaluated **on the backend** with `_` bound to the live handle behind the proxy, so descent works without ever serializing the handle itself.

`globals-touched` (§9.6) is recorded lazily by instrumented global reads/writes exactly as §9.6 describes, and serialized like any value (proxies as needed). `catch-stack` (§10.2) is serialized as frame descriptors.

4.5 R-III.5 Wire messages

A framed envelope with a `kind` discriminator. App→debugger preserves §8.2 and adds nothing semantically; debugger→app preserves §8.3 and **adds the inspection/ eval/breakpoint-table operations that were free synchronous CL in-image**.

Backend → alfe (extends §8.2):

- `:hit` fid form-id when bp-kind snapshot (steady | volatile | trace)
- `:unhandled-error` snapshot error-string errno (§10.1; +ERRNO)
- `:caught-error` snapshot error-object errno (§10.2)
- `:thread-exit` value-or-error
- `:output` text (interleaved princ/prompt)
- `:reply` id payload (answer to an inspection cmd)

alfe → backend (extends §8.3):

- `:continue` | `:continue-with-return` value | `:abort` (§10.1)
- `:set-volatile` ((fid form-id when) …) then implicit continue (stepping, §6)
- `:add-breakpoint` fid form-id when &key steady cond-body action (was: direct table write, §12; now a message that updates the backend table + Bloom bit)

- `:remove-breakpoint` fid form-id when
- `:eval-in-frame` id frame-index form (was: synchronous CL; now RPC)
- `:set-binding` frame-index symbol new-value (§9.4/§9.5; validated)
- `:inspect-accessor` id proxy-or-value accessor-form (inspector descent, R-IV)
- `:list-globals` id (§9.6 on demand)
- `:interrupt` (sets a cooperative-interrupt flag)

`REQ-P1:` every message length-prefixed or sentinel-delimited. `REQ-P2:` every inspection command carries a correlation `id` echoed in `:reply`. `REQ-P3:` alfe tolerates `:output` at any time. The framing reuses the parent's Emacs-UI RPC shape (§20.1: line-oriented S-expressions read with `read`) — now on the **debugger application** channel, which the parent never had.

4.6 R-III.6 Breakpoint conditions and trace actions over the wire (§2, §6.4)

The parent allows a breakpoint *condition* to be "a CL form evaluated in the debugger thread, with access to the application's live variables and to debugger workspace slots (§16.2)." Remotely this splits:

- **Access to live application variables** the condition must run **on the backend** (that is where the variables are) and must be cheap (it is tested at every pass of that poll point). So a remote condition is an **AutoLISP** form, instrumented and installed on the backend; the breakpoint fires only if it returns non-`nil`. No round-trip unless it fires.
- **Access to workspace slots `$N`** `$N` lives in alfe, not the backend. When the breakpoint is set, alfe **substitutes each `$N` by its current value** (serializing it; an opaque `$N` becomes a backend proxy handle valid for the session) and bakes it into the AutoLISP condition. **Semantic note:** a remote condition's workspace references are snapshotted at set-time, not re-resolved live — a deliberate, documented divergence from the in-image semantics.

Trace actions (§6.4) keep the parent's key property that **the application never runs the print code** (so tracing `print` internals is safe). A trace point fires backend sends a lightweight `:hit` of kind `trace` with the values the trace form needs in the snapshot **alfe** evaluates the trace form against the serialized snapshot, prints to the trace stream, and immediately replies `:continue`. This costs one round-trip per trace hit (vs a queue hop in-image); acceptable, and it preserves "no application-side printing, no encapsulation."

4.7 R-III.7 Error handling over the wire (§10)

- **`*error*` interception (§10.1).** `aldbg:` installs a debugger-aware `*error*`. On entry it builds a snapshot from the shadow registry, captures `(getvar "ERRNO")` (because `vl-catch-all-error-message` text is **localized to the installed product language** and unsuitable for programmatic dispatch), sends `:unhandled-error`, and blocks. The three resolutions of §10.1 map as:
  - `:continue-with-error` → call the user's real `*error*`.
  - `:continue-with-return` → **restricted**. Safe only for an error raised *inside a form we instrumented* (we can supply a value at that poll point). For errors from CAD/host primitives it is **not** offered (the parent already warns this for host primitives; remotely nearly all primitives are host).
  - `:abort` → the backend's top-level `vl-catch-all-apply` wrapper unwinds the evaluation and restores system variables, then emits `:thread-exit`.
- **`vl-catch-all-apply` (§10.2).** The instrumented wrapper pushes a catch frame onto `aldbg:*catch-stack*`; with "break on caught error" enabled it sends `:caught-error` *before* returning the AutoLISP error object, then blocks for a command. Default off, exactly as §10.2.
- **No condition system (§10.3)** — unchanged; the AutoLISP-facing UI offers no restarts; `clautolisp.debug.remote` uses the host condition system internally in alfe, invisibly.

5 R-IV. The inspector over the wire

Part IV is preserved. The only change: the inspector becomes **lazy and remote**.

- `inspect-page-for` (§15) for a **serializable** value runs in alfe on the inlined value. For a **proxy** value, the components and their previews are obtained by evaluating the accessors on the backend: a descent into component `i` sends `:inspect-accessor id proxy accessor-form` and renders the `:reply`.
- The accessor S-expressions of §15.1 (`(cdr (assoc g (entget _)))`, `(ssname _ n)`, `(vla-get-prop _)`, …) are **exactly** the right thing to send, because they are designed to be re-evaluated with `_` = the parent's path expression. Remotely, `_` resolves to the live handle behind the proxy on the backend. This is the cleanest fit in the whole reconciliation: the inspector's accessor design *already assumed* re-evaluatable expressions.
- `session-path-expression` (§15.2) composes unchanged. The `:opaque` case (§15.1, §15.2) generalises: a value that **cannot be serialized and has no AutoLISP accessor** is `:opaque`; descent is still allowed against its backend proxy, and `session-path-expression` returns `(values partial :partial)` and offers a workspace binding (6).
- Latency: each descent into a proxy is a round-trip. UIs should render the parent value immediately and fill components as replies arrive (§R-V).

6 R-IV-b. The workspace `$N` over the wire (§16.2)

The workspace stays entirely debugger-side, as §16.2 mandates ("resolved by the debugger's eval layer, not by AutoLISP's evaluator"). Remotely this is *cleaner*: `$N` never reaches the backend except when its **value** is needed in a backend eval, in which case alfe substitutes the value (serializing it; an opaque `$N` substitutes its backend proxy handle, valid for the session). `session-bind` (§16.1) destinations map as:

- `:workspace` — pure alfe; the value (or proxy) is held in alfe.
- `:global SYMBOL` / `:setq SYMBOL` — a `:eval-in-frame` of `(setq SYMBOL v)` on the backend, `v` serialized or proxy.
- `:frame FRAME SYMBOL` — a `:set-binding` (4.5); §16.1's restriction (no *new* frame-local binding mid-execution) is enforced on the backend.

7 R-V. UI deltas

Part V is unchanged at the §17 API level; a parent-conformant UI runs as-is. Only operational notes apply:

- **Asynchrony.** In-image, `cmd-eval` returned synchronously (§8.3). Remotely it is a round-trip; UIs already issue `cmd-*` from the UI thread and receive `ui-*` notifications, so the library presents the same surface and resolves the RPC under the hood. UIs should not assume sub-millisecond eval latency.
- **Output.** `:output` (princ/prompt from running code) is surfaced via `ui-show-message :info` or a dedicated output area; it can arrive between a `:hit` and the next command (§R-III.5, REQ-P3).
- **Source overlays (§20.2, §19.1).** Poll-point and breakpoint markers are placed from alfe's debug metadata (which alfe holds, R-II.5); the backend is never asked for source positions.
- **Disconnect.** If the wire drops while paused, the UI shows the session as `:disconnected`; reconnect re-attaches to the live backend pause if the backend is still blocked, else reports the application as lost.

8 R-VI. Sessions and lifecycle deltas

- **One application thread (§1.2, §23).** The backend is the CAD main thread: exactly one application "thread" per backend connection. The parent's per-thread breakpoint scoping — needed in-image so the debugger thread could run the same code (e.g. the printer) without a global-breakpoint deadlock (§23) — is **unnecessary** here, because the debugger is a separate process and never executes the application's code. Breakpoints are simply per-session.
- **Multiple documents.** Debugging across several open CAD documents = several sessions/connections (one `aldbg:` state per document namespace, 9); not multiple threads in one session.
- **Start/attach (§21, §22).** `start-session :ui …` attaches a UI and opens the backend connection; the calling alfe thread is the debugger thread. `attach-thread` becomes "attach to a backend already running instrumented code": set `aldbg:*debug-flag*` at the next poll point. The §22 caveat (attachment to a thread inside an *ordinary* body

takes effect only on return to a debug frame) maps to "code currently running `~ord` bodies is not stoppable until it re-enters a `~dbg` body" — the price of the two-bodies design, unchanged.

- **Teardown (§24).** On session end `aldbg:` clears breakpoints and the Bloom bits, restores the original `*error*` path, swaps public symbols back to `~ord` bodies, clears the shadow registry, and drops proxy tables; alfe discards the workspace. `:global=/`:setq= writes persist in the CAD global namespace, the user's responsibility (§24).

9 R-VII. Namespaces and product specifics

- **Where backend state lives.** AutoLISP globals are per-document. `aldbg:` state is installed in the document namespace of the debugged drawing. Cross-document items (rare) use the blackboard namespace (`vl-bb-set=/=vl-bb-ref`). A separate-namespace VLX has isolated globals; `aldbg:` and the instrumented bodies must be installed *inside* that namespace or the `aldbg:` calls in the debug bodies will not resolve.

- **BricsCAD vs AutoCAD.** BricsCAD LISP is largely AutoLISP-compatible; load `vle-extension.lsp` for cross-product helpers. Confine product-specific surface (the `defun-q-list-*` accessors, `vlax-*` details, `MILLISECS=/timing sysvars) to a small backend shim selected at session start. The =aldbg:` runtime depends only on widely-available primitives: `vl-catch-all-apply` and friends, `logand=/=logior=/=lsh`, `read=/=eval`, `getvar "ERRNO"`, and basic list ops. It must **not** depend on the in-process IDE debuggers (VLIDE is Windows-only; BLADE's hooks are not exposed to a remote client).

10 R-VIII. Conformance delta (amends Parts VII §25, §26)

Per [RFC2119]. These **amend** §25/§26 for the alfe deployment.

Backend (`aldbg:`) MUST:

1. Implement `aldbg:pp` with the §11.1 fast path + §11.2 Bloom filter using alfe-assigned function-ids, consulting **local** tables with no wire traffic on non-firing poll points (R-III.2).

2. Maintain the shadow call/binding stack so a snapshot reproducing §9.2/§9.3/ §9.4 can be built and serialized, including shadowed-binding chains and shadowed-binding writes (R-III.4).

3. Serialize values per R-III.4.2, emitting proxies for ename, selection set, file descriptor, and VLA object, and resolving accessors against live handles on demand (R-IV).

4. Implement the pause loop and the debug sub-protocol (R-III.3, R-III.5), including `:eval-in-frame`, `:set-binding`, `:inspect-accessor`, breakpoint table updates, and volatile-breakpoint stepping.

5. Intercept `*error*` and record `vl-catch-all-apply` frames when debugging is active, capturing `ERRNO` (R-III.7), and guard reactor reentrancy (REQ-T3).

alfe (`clautolisp.debug.remote`) MUST:

1. Implement the §12 and §17 APIs unchanged in signature, backed by the wire.

2. Perform all instrumentation in alfe (R-II), emitting `~ord=/=~dbg` AutoLISP bodies and holding all §7 debug metadata; the backend MUST NOT parse user source.

3. Compute stepping placements (§6) in alfe and ship them as volatile breakpoints (R-III.5), so no per-step round-trips occur beyond the resume.

Relaxed / changed from the parent:

1. §25.1 (reader attaches positions) applies to alfe's reader; the backend has no such duty.

2. The §8.3 guarantee of **synchronous, message-free** snapshot inspection is **withdrawn**; inspection is RPC (R-III, §0.1). The deferred async `:eval-result` becomes mandatory.

3. The parent §23 multi-thread/per-thread-breakpoint requirements do not apply; one application thread per session (R-VI).

Transport MUST satisfy REQ-T1..T3 and REQ-P1..P3. A transport failing REQ-T1 restricts conformance to **post-mortem mode** (break-on-error + reconstructed backtrace; no live breakpoints/stepping), which MUST be advertised at attach via `*protocol-version*` capability flags (§27).

11 R-IX. Feature-fidelity matrix (keyed to parent sections)

Parent feature	§	Remote fidelity	Mechanism / note
Two bodies / omnipresence	5	Full	`~ord=/=~dbg` + source-leve
Poll points, dense form-ids	5.3,11	Full	`aldbg:pp`, alfe-assigned fid
Bloom-filter fast path	11.2	Full	AutoLISP bitset; more imp
Steady & volatile breakpoints	2,12	Full	backend table; updated by
Stepping in/out/over/call/finish	6	Full	placements computed in al
Advance to point	6.2	Full	one volatile bp + continue
Conditional breakpoints	2	Partial	AutoLISP condition; `$N` sn
Tracing (no app-side print)	6.4	Full*	round-trip per trace hit; pr
Backtrace / call stack	9.3	Full	shadow registry
visible-names / binding stack	9.2,9.4	Full	shadow registry
Show & write **shadowed** bindings	9.4	Full	registry holds saved values
eval-in-frame (innermost)	17.2	Full	RPC `:eval-in-frame`
eval-in-frame (outer, with-frame-binds)	9.3	Partial	backend re-binds from regis
set-variable / set-binding	9.4,9.5	Full	write-safe (AutoLISP dyna
Inspector pages + accessors	15	Full	accessors evaluated on back
Path-expression composition	15.2	Full	unchanged; opaque proxy/v
Workspace `$N`	16.2	Full	alfe-side; value/proxy subst
error interception	10.1	Full	`aldbg: *error*` + ERRN
continue-with-return	10.1	Partial	instrumented-form errors o
vl-catch-all-apply frames / break-on	10.2	Full	`aldbg:*catch-stack*`
Three UIs	V	Full	API unchanged; latency/as
Multiple application threads	23	None/N-A	single backend thread; mul
Synchronous message-free inspection	8.3	None (by design)	becomes RPC (§0.1)
Step into SUBR / FAS / VLX (no source)	VIII	None	nothing to instrument
Preemptive interrupt of tight loop	–	Partial	cooperative, at poll points

12 R-X. Limitations and non-goals specific to a native backend

1. **No instrumentation of opaque code.** Built-in `SUBR=s, =vlax=/ActiveX methods, and =FAS=/=VLX` without source cannot be given debug bodies; the debugger steps *over* them. Wrap them in `vl-catch-all-apply` for break-on- error context only.

1b. **Only alfe-loaded code is debuggable** (3.3). Code the CAD loaded it-

self (`ACADDOC.lsp`, `APPLOAD=/Startup Suite, =autoload`, `VLX/FAS`, `S::STARTUP`, native `(load …)`) was never instrumented; to debug a function the programmer MUST re-load its source through alfe (`REQ-LOAD1`), which then shadows the native definition under the public name for the session. Debuggingness propagates only across the alfe-instrumented sub-graph (`REQ-LOAD2`); it stops at the boundary of natively-loaded callees.

1. **Cooperative interruption only.** A running program is stoppable only at a poll point; `:interrupt` sets a flag honoured at the next `aldbg:pp`. To keep tight loops interruptible, the instrumenter SHOULD place a poll point at each iteration of `while=/=repeat=/=foreach` bodies.

2. **Editor-input interaction.** If paused while a `command=/=getXXX` awaits input in the drawing editor, the pause loop and the prompt compete for the user. Do not single-step *into* an input-awaiting `command`; treat the whole `command` form as one step.

3. **Overhead and stack depth.** Debug bodies are materially slower and add frames; deep recursion may overflow. Hence two bodies and configurable poll-point granularity (statement default; expression optional).

4. **Localized error text.** Dispatch on `ERRNO`, never on `vl-catch-all-error-message` text.

5. **Reactor reentrancy** (REQ-T3) is guarded but reactors that themselves run user LISP during a pause remain a sharp edge.

6. **Conditions snapshot workspace at set-time** (R-III.6), unlike the live in-image semantics.

7. **Outer-frame eval** (with-frame-bindings) temporarily perturbs the binding state on the backend; offered best-effort, always restored, never during a reactor callback.

8. The parent's `Part VIII` deferral of remote debugging is **resolved by this addendum**; its other deferrals (cross-function inlining, instruction-level stepping, persistent breakpoints, record-and-replay) remain deferred.

13 R-XI. Phased implementation plan

1. **Transport + framing** (REQ-T1..T3, P1..P3). Prove a pause nests inside an in-flight `:eval` and that one `:eval-in-frame` round-trips. Reuse the §20.1 S-expr RPC shape.

2. **`aldbg:` core:** `*debug-flag*`, breakpoint table, Bloom filter, `aldbg:pp`, `aldbg:enter/exit`, pause loop. Hand-write one `~dbg` function to validate hooks, snapshot, and backtrace before the instrumenter exists.

3. **alfe instrumenter + `autolisp-compile[-file]`** (R-II): §3 reader, §5/§7 metadata, `~ord=/=~dbg` emission, Fjeld propagation, install on backend.

4. **Breakpoints + stepping + backtrace + innermost eval/inspect** end-to-end (the "Full" rows).

5. **Inspector over the wire** (R-IV): proxies, accessor evaluation, path expressions, workspace substitution.

6. **Error integration** (R-III.7): `*error*`, `vl-catch-all`, ERRNO, break-on- caught, continue-with-return for instrumented forms.

7. **Shadowed-binding read/write, outer-frame eval, conditions, tracing, cooperative interrupt** (the "Partial" rows), then hardening (reactors, command-input, namespaces, BricsCAD/AutoCAD shim).

14 R-XII. Open questions to reconcile with the parent author

1. **Does clautolisp itself execute anything in the alfe deployment, or is it only the reader/instrumenter/debugger host?** This addendum assumes the **backend native engine** executes the program and clautolisp provides the reader, instrumenter, inspector, and debugger library in alfe. If instead alfe is meant to evaluate non-CAD forms locally in clautolisp and forward only CAD-side-effecting primitives to the backend (a split evaluator), the snapshot and binding model become a hybrid and Part R-III needs a second half. Confirm.

2. **Is per-call-site two-body dispatch (§5.1) required, or is "install `~dbg` under the public name while a session is active"**

(R-II.2) acceptable? The latter is simpler and justified by the single backend thread; confirm no requirement (e.g. a concurrently-running production command on the same document) forces true per-site dispatch.

3. **Transport:** does alfe's existing alfe CAD protocol satisfy REQ-T1 (synchronous mid-evaluation readability)? If not, is post-mortem mode (R-VIII §11/capability flag) acceptable for v1, with live stepping deferred until a second control channel exists?

4. **Conditions:** is set-time workspace snapshotting (R-III.6) acceptable, or must workspace references in conditions resolve live (forcing a round-trip per poll-point pass and so, effectively, banning conditions on hot poll points)?

5. **continue-with-return** (§10.1) remote restriction to instrumented-form errors: acceptable, or should it be removed entirely remotely to avoid the footgun?